

06.01: Colecciones, POO y Ficheros

Contenido

1. Introducción	2
2. Ejemplo comentado: Lista de estudiantes.....	2
3. Mención breve: CSV	4
4. Ejercicio de Asimilación.....	5
5. Ejercicio de Consolidación	6

1. Introducción

En programación, los datos se almacenan y manipulan mediante **colecciones** (listas, diccionarios, etc.) y **objetos** definidos con **POO (Programación Orientada a Objetos)**.

Para que estos datos no se pierdan al cerrar el programa, se pueden guardar en **ficheros**, y uno de los formatos más utilizados es **JSON**, que permite almacenar datos de manera estructurada y legible.

Conceptos clave:

- **Colecciones:** permiten agrupar múltiples elementos, sean valores simples u **objetos**.
- **Objetos:** encapsulan atributos y comportamientos; pueden almacenarse en colecciones.
- **JSON:** formato de texto para representar datos; guarda información estructurada de manera que Python puede reconstruir objetos o colecciones fácilmente.
- **Flujo de datos completo:**
 - **Objeto → Colección → Fichero JSON → Leer → Reconstruir objeto**

2. Ejemplo comentado: Lista de estudiantes

Ejemplo:

```
import json
```

```
# Definición de la clase Estudiante
```

```
class Estudiante:
```

```
    def __init__(self, nombre, edad, nota):
```

```
        self.nombre = nombre
```

```
        self.edad = edad
```

```
        self.nota = nota
```

```
# Método para convertir el objeto en diccionario
```

```
def to_dict(self):
```

```
    return {"nombre": self.nombre, "edad": self.edad, "nota": self.nota}
```

```

# Representación legible
def __str__(self):
    return f"{self.nombre}, {self.edad} años, nota: {self.nota}"

# Crear una lista de objetos Estudiante
alumnos = [
    Estudiante("Ana", 20, 8.5),
    Estudiante("Luis", 19, 7.0),
    Estudiante("Carlos", 21, 9.2)
]

# Guardar la lista de objetos en un fichero JSON
with open("alumnos.json", "w") as f:
    json.dump([alumno.to_dict() for alumno in alumnos], f, indent=4)

# Leer el fichero JSON y reconstruir los objetos
with open("alumnos.json", "r") as f:
    datos = json.load(f)

alumnos_recuperados = [Estudiante(d["nombre"], d["edad"], d["nota"]) for d in datos]

# Mostrar los objetos reconstruidos
for alumno in alumnos_recuperados:
    print(alumno)

```

Qué se muestra:

- Creación de objetos y almacenamiento en lista.
- Conversión de objetos a diccionario para JSON.
- Guardado en fichero JSON (json.dump).
- Lectura desde fichero y reconstrucción de objetos (json.load).
- Iteración sobre la lista reconstruida.

3. Mención breve: CSV

Aunque JSON es más flexible y moderno, también es posible almacenar datos en **CSV** (valores separados por comas), especialmente si se quiere usar los datos en hojas de cálculo.

Ejemplo:

```
import csv
```

```
# Guardar datos en CSV
```

```
with open("alumnos.csv", "w", newline="") as f:
```

```
    writer = csv.writer(f)
```

```
    writer.writerow(["Nombre", "Edad", "Nota"])
```

```
    for alumno in alumnos:
```

```
        writer.writerow([alumno.nombre, alumno.edad, alumno.nota])
```

CSV es útil para **importar y exportar** datos tabulares, pero JSON es más apropiado cuando trabajamos con **objetos y estructuras anidadas**.

4. Ejercicio de Asimilación

Objetivo

Practicar la creación de objetos, su almacenamiento en colecciones y la persistencia en ficheros, además de aplicar filtrado y estadísticas simples.

Enunciado:

1. Define la clase Estudiante con los atributos:
 - nombre (str)
 - edad (int)
 - nota (float)
 - grupo (str)
2. Crea una lista de al menos 3 estudiantes en el código.
3. Lee un fichero JSON opcional alumnos_extra.json que contenga estudiantes adicionales (puede no existir).
 - Si el fichero existe, agrega los estudiantes leídos a la lista.
4. Guarda la lista completa de estudiantes en un fichero JSON.
5. Reconstruye los objetos desde el fichero JSON y muéstralos en pantalla.
6. Filtra y muestra los estudiantes con **nota $\geq$ 8**.
7. Calcula y muestra la **nota media** de todos los estudiantes.
8. Exporta todos los estudiantes a un fichero CSV alumnos_completo.csv.

5. Ejercicio de Consolidación

Objetivo

Trabajar de manera integrada con **POO, colecciones y ficheros**, aplicando lectura/escritura de JSON y CSV, filtrado y análisis de datos en un contexto distinto al de estudiantes.

Enunciado:

1. Crea la clase Producto con los atributos:
 - nombre (str)
 - categoria (str)
 - precio (float)
 - stock (int)
2. Supón que tienes **varios ficheros JSON** de productos por categoría:
 - electronica.json
 - alimentacion.json
 - hogar.json

Cada archivo contiene una lista de productos en formato JSON.

3. El programa deberá:
 - Leer todos los ficheros JSON (ver la nota al final del ejercicio) y **reconstruir los objetos** Producto.
 - Combinar todos los productos en **una sola lista**.
 - Mostrar todos los productos en pantalla.
 - Filtrar y mostrar únicamente los productos que cumplan ambas condiciones:
 - Precio ≥ 50
 - Stock ≥ 10
 - Calcular y mostrar el **precio medio de todos los productos**.
 - Crear un nuevo JSON productos_destacados.json con los productos filtrados.
 - Exportar todos los productos a un fichero CSV todos_productos.csv.

Nota sobre la lectura de varios archivos JSON:

Para poder leer varios archivos JSON automáticamente, es necesario usar una función que liste los archivos en la carpeta. Puedes hacerlo de dos formas distintas:

Opción 1: Usando glob (más directa):

```
import glob  
for archivo in glob.glob("*.json"): # busca todos los archivos .json del directorio actual
```

Opción 2: Usando os

```
import os  
for archivo in os.listdir(): # lista todos los nombres de archivos y carpetas  
    if archivo.endswith(".json"): # selecciona solo los que terminan en .json
```